

# Tarea S2.01. Nociones básicas SQL

## Contenido

Nivel 1 ..... 1

    Ejercicio 1:..... 1

    Ejercicio 2: ..... 2

    Ejercicio 3..... 5

    Ejercicio 4..... 7

    Ejercicio 5..... 10

    Ejercicio 6..... 11

    Ejercicio 7 ..... 13

    Ejercicio 8..... 14

    Ejercicio 9..... 15

    Ejercicio 10..... 16

Nivel 2 ..... 17

    Ejercicio 1 ..... 17

    Ejercicio 2 ..... 18

    Ejercicio 3..... 19

    Ejercicio 4..... 20

    Ejercicio 5..... 21

Nivel 3 ..... 22

    Ejercicio 1 ..... 22

    Ejercicio 2..... 23

# Nivel 1

## Ejercicio 1:

A partir de los documentos adjuntos (estructura\_datos y datos\_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

Tablas en el dataset.

|   |                        |
|---|------------------------|
|   | Tables_in_transactions |
| ▶ | company                |
|   | transaction            |

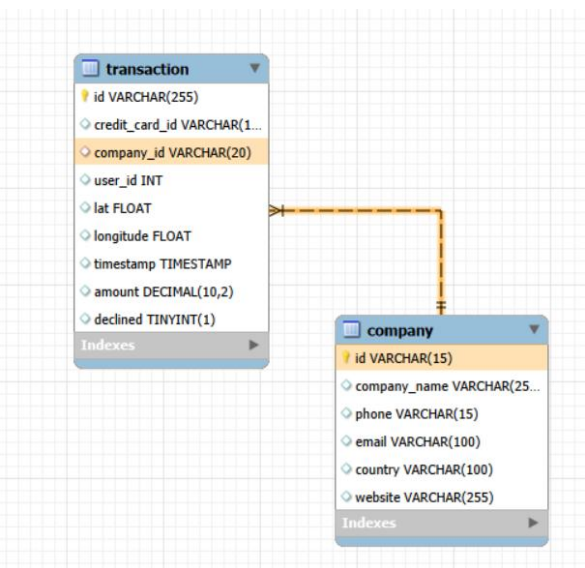
Tabla *company*:

|   | id     | company_name              | phone          | email                            | country   | website                      |
|---|--------|---------------------------|----------------|----------------------------------|-----------|------------------------------|
| ▶ | b-2222 | Ac Fermentum Incorporated | 06 85 56 52 33 | donec.porttitor.tellus@yahoo.net | Germany   | https://instagram.com/site   |
|   | b-2226 | Magna A Neque Industries  | 04 14 44 64 62 | risus.donec.nibh@icloud.org      | Australia | https://whatsapp.com/group/9 |

Tabla *transaction*:

| id                                   | credit_card_id | company_id | user_id | lat     | longitude | timestamp           | amount | declined |
|--------------------------------------|----------------|------------|---------|---------|-----------|---------------------|--------|----------|
| 00043A49-2949-494B-A5DD-A5BAE38B19DD | CcS-9294       | b-2458     | 4713    | 46.1999 | 1.43554   | 2024-08-28 07:16:46 | 395.43 | 0        |
| 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD | CcS-5019       | b-2370     | 438     | 41.5972 | 12.2218   | 2016-12-21 20:07:18 | 155.63 | 0        |

Diagrama relacional:



La base de datos de transacciones creada consta de dos tablas: empresa y transacción. El modelo establece una relación de uno a muchos entre las tablas, donde una empresa puede tener múltiples transacciones. La clave primaria de esta relación es el ID de cada empresa. Cada empresa tiene los siguientes atributos: Nombre, Número de teléfono, Correo electrónico, País, Sitio web, y cada transacción tiene los siguientes atributos: ID de tarjeta de crédito, ID de usuario, Latitud, Longitud, Marca de tiempo, Valor de la transacción y Status de Rechazada o no.

## Ejercicio 2:

Utilizando JOIN realizarás las siguientes consultas:

- 1- Listado de los países que están generando ventas.

The screenshot shows a database query tool interface. At the top, there is a toolbar with various icons and a dropdown menu set to 'Limit to 5000 rows'. Below the toolbar, the SQL query is entered in a text area:

```
1 SELECT DISTINCT country FROM company
2 JOIN transaction
3 ON company.id = transaction.company_id
4 WHERE transaction.declined = 0;
```

Below the query editor, there is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid displays a list of countries:

| country        |
|----------------|
| Netherlands    |
| Sweden         |
| Ireland        |
| United States  |
| Belgium        |
| Germany        |
| Norway         |
| France         |
| United Kingdom |
| New Zealand    |
| Italy          |
| Australia      |
| China          |
| Spain          |
| Canada         |

At the bottom, there is an 'Output' section with a dropdown menu set to 'Action Output'. It displays a log of actions:

| #     | Time     | Action                                                                                             | Message            |
|-------|----------|----------------------------------------------------------------------------------------------------|--------------------|
| ✓ 130 | 12:36:44 | SELECT DISTINCT country FROM company JOIN transaction ON company.id = transaction.company_id WH... | 15 row(s) returned |
| ✓ 131 | 12:36:45 | SELECT DISTINCT country FROM company JOIN transaction ON company.id = transaction.company_id WH... | 15 row(s) returned |

Para listar solo los países que generan ventas, se utiliza una unión (JOIN) entre las tablas Company y Transaction a través de las columnas id y company\_id, respectivamente, utilizando como condición que la empresa tenga al menos un valor igual a cero en la columna declined, lo que significa que tiene al menos una transacción válida.

2- Desde cuántos países se generan las ventas.

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search, along with a 'Limit to 5000 rows' dropdown. The query editor contains the following SQL code:

```
1 SELECT COUNT(DISTINCT country) AS Total_Paises_con_Ventas
2 FROM company
3 JOIN transaction
4 ON company.id = transaction.company_id
5 WHERE transaction.declined = 0;
```

Below the query editor is the 'Result Grid' section, which displays the query results in a table:

| Total_Paises_con_Ventas |
|-------------------------|
| 15                      |

At the bottom, the 'Output' section shows the execution log. It includes a table with columns for '#', 'Time', 'Action', and 'Message':

| #     | Time     | Action                                                                                             | Message            |
|-------|----------|----------------------------------------------------------------------------------------------------|--------------------|
| ✓ 131 | 12:36:45 | SELECT DISTINCT country FROM company JOIN transaction ON company.id = transaction.company_id WH... | 15 row(s) returned |
| ✓ 132 | 12:37:52 | SELECT COUNT(DISTINCT country) AS Total_Paises_con_Ventas FROM company JOIN transaction ON co...   | 1 row(s) returned  |

Para determinar el número total de países que generan ventas, se utiliza la misma cláusula JOIN de la consulta anterior, y para realizar el recuento, se utiliza COUNT(DISTINCT country).

3- Identifica a la compañía con la mayor media de ventas.

The screenshot shows a SQL query editor with the following query:

```
1 • SELECT company.company_name, ROUND(AVG(amount),2) AS media_de_ventas FROM company
2 JOIN transaction
3 ON company.id = transaction.company_id
4 WHERE transaction.declined = 0
5 GROUP BY company.id
6 ORDER BY media_de_ventas DESC
7 LIMIT 1;
```

Below the query editor, the 'Result Grid' shows the results of the query:

| company_name              | media_de_ventas |
|---------------------------|-----------------|
| Ac Fermentum Incorporated | 288.35          |

At the bottom, the 'Output' section shows the execution details:

| # | Time     | Action                                                                                      | Message           |
|---|----------|---------------------------------------------------------------------------------------------|-------------------|
| 1 | 11:54:15 | SELECT company.company_name, ROUND(AVG(amount),2) AS media_de_ventas FROM company JOIN t... | 1 row(s) returned |

Para encontrar la empresa con el promedio de ventas más alto entre todas las listadas, se utiliza inicialmente la misma estructura JOIN empleada en consultas anteriores, seguida de la aplicación de la función `AVG(amount)`, que calcula el promedio de todas las ventas realizadas. A continuación, las empresas se agrupan por id y, para encontrar la empresa con el promedio de ventas más alto, la columna de promedio de ventas se ordena en orden descendente, mostrando únicamente la fila con el valor más elevado.

## Ejercicio 3

Utilizando sólo subconsultas (sin utilizar JOIN):

- 1- Muestra todas las transacciones realizadas por empresas de Alemania.

```
1 • SELECT COUNT(id) AS total_de_transacciones
2 FROM transaction
3 WHERE company_id IN (SELECT id
4 FROM company
5 WHERE country = "Germany")
6 AND declined = 0;
7
```

| Result Grid |                        | Filter Rows: | Export: | Wrap Cell Content: |
|-------------|------------------------|--------------|---------|--------------------|
|             | total_de_transacciones |              |         |                    |
| ▶           | 5521                   |              |         |                    |

| Result 12 x   |          |                                                                                                  |                   |
|---------------|----------|--------------------------------------------------------------------------------------------------|-------------------|
| Output        |          |                                                                                                  |                   |
| Action Output |          |                                                                                                  |                   |
| #             | Time     | Action                                                                                           | Message           |
| ✓ 1           | 13:02:41 | SELECT COUNT(id) AS total_de_transacciones FROM transaction WHERE company_id IN (SELECT id FR... | 1 row(s) returned |

Para encontrar el número total de transacciones solo para empresas en Alemania, puede utilizar una subconsulta que le permita filtrar el `company_id` solo para las empresas cuyo país se llama Alemania y que tienen al menos una transacción válida.

- 2- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```
1 • SELECT amount, (  
2     SELECT company_name  
3     FROM company co  
4     WHERE co.id = t.company_id AND t.declined = 0  
5     ) AS Company_name  
6 FROM transaction t  
7 WHERE amount > (SELECT AVG(amount) FROM transaction);
```

Result Grid

| amount | Company_name                      |
|--------|-----------------------------------|
| 395.43 | Eget Tincidunt Dui Institute      |
| 326.01 | Neque Tellus Imperdiet Corp.      |
| 294.59 | Aliquam Iaculis Lacus Corp.       |
| 383.63 | Lorem Ipsum Dolor Corp.           |
| 339.94 | Nulla Integer Vulputate Corp.     |
| 369.71 | Auctor Mauris Corp.               |
| 271.87 | Nunc Ac PC                        |
| 316.90 | Ac Fermentum Incorporated         |
| 425.10 | Non Magna LLC                     |
| 355.92 | Cras Vehicula Aliquet Industries  |
| 366.92 | Maecenas Malesuada Fringilla I... |
| 354.07 | Enim Condimentum Ltd              |

Result 4 x

Output

Action Output

| # | Time     | Action                                                                                     | Message               |
|---|----------|--------------------------------------------------------------------------------------------|-----------------------|
| 1 | 12:56:20 | SELECT amount, ( SELECT company_name FROM company co WHERE co.id = t.company_id AND t.d... | 19171 row(s) returned |

Puedes realizar una consulta utilizando dos subconsultas, la primera para unir la columna company\_name de la tabla transaction y filtrar por nombre de empresa, y la segunda para comparar el valor de la transacción con el promedio total de todas las ventas. De esta forma, obtenemos una tabla solo con los nombres de las empresas cuyas ventas totales superan el promedio.

- 3- Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
1 • SELECT id, company_name
2   FROM company co
3  WHERE id IN (
4      SELECT company_id
5      FROM transaction t
6      WHERE t.company_id = co.id AND declined = 1
7  )
8  AND id NOT IN (
9      SELECT company_id
10     FROM transaction t
11     WHERE t.company_id = co.id AND declined = 0
12  );
13
```

The result grid shows two columns: 'id' and 'company\_name'. Both cells contain 'NULL'.

Below the result grid, there is a tab labeled 'company 7' and an 'Output' section. The 'Action Output' section shows a message: '1 12:13:16 SELECT id, company\_name FROM company co WHERE id IN ( SELECT company\_id FROM transac... 0 row(s) returned'.

Para encontrar empresas sin transacciones válidas, se puede usar la cláusula WHERE para evaluar dos condiciones. La primera condición se basa en una subconsulta que filtra las empresas cuyo valor en la columna declined es distinto de 0. La segunda condición es una segunda subconsulta que filtra las empresas cuyo valor en la columna declined es igual a 1. Un valor de 1 indica que la transacción fue rechazada, y un valor de 0 que fue aceptada.

## Ejercicio 4

Tu tarea es diseñar y crear una tabla llamada credit\_card que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos\_introducir\_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo:

Se ha diseñado y creado la tabla credit\_card para almacenar la información de las tarjetas de crédito utilizadas en las transacciones. Esta tabla contiene los campos id, iban, pan, pin, cvv y expiring\_date, siendo el campo id la clave primaria que identifica de forma única cada tarjeta.

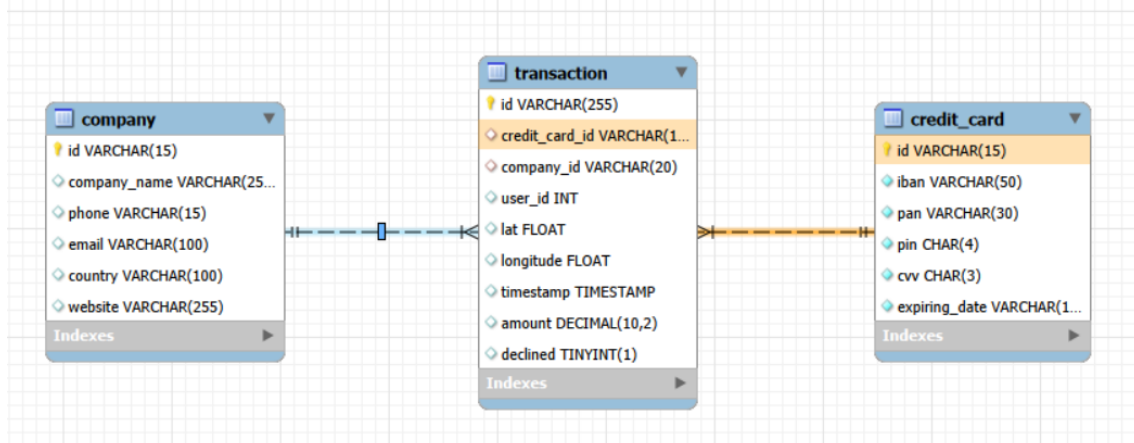
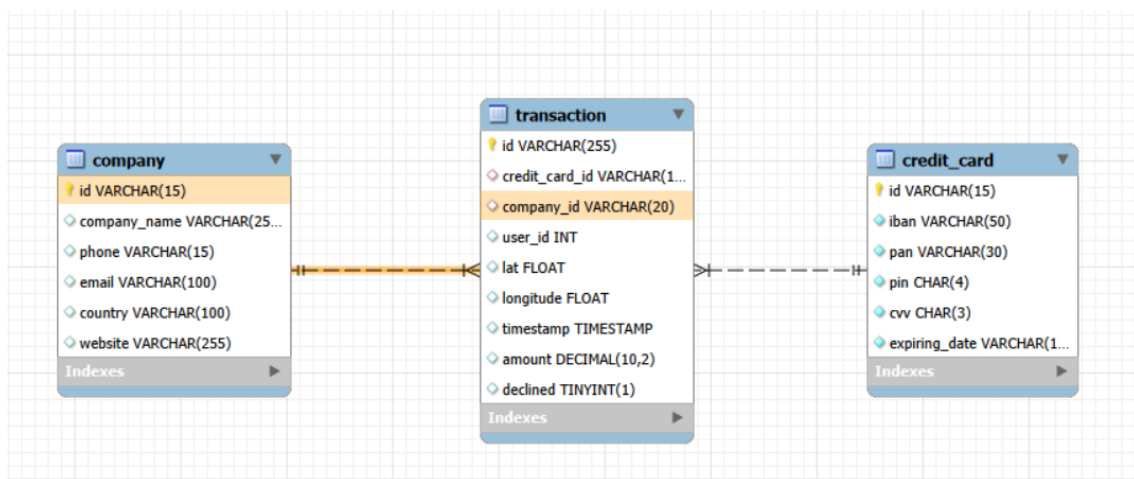
La tabla transaction se relaciona con credit\_card mediante el campo credit\_card\_id, que actúa como clave foránea y referencia al campo id de la tabla credit\_card. Esta relación es de uno a muchos (1:N), ya que una misma tarjeta de crédito



puede utilizarse en varias transacciones, mientras que cada transacción está asociada a una única tarjeta.

Asimismo, la tabla transaction mantiene una relación con la tabla company mediante el campo company\_id, que también es una clave foránea. De esta forma, cada transacción queda vinculada tanto a la empresa donde se realizó como a la tarjeta de crédito utilizada.

Este diseño mejora la organización de la base de datos, evita la duplicación de información y garantiza la integridad referencial entre las tablas.



Nivel 1 - 4    N1-Ex 4 - datos\_introducir\_cre...    SQL File 7\*    credit\_card

Limit to 50000 rows

```

1 -- Insertamos datos de credit_card
2 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633', '325'
3 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO26854763748537475216568689', '5142423821948828', '9
4 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598',
5 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '6
6 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900'
7 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT87806228135092429456346', '544 58654 54343 384', '8
8 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5075',
9 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE89681434837748781813', '3763 747687 76666', '2298',
10 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH62714428368066765294', '344283273252593', '7545', '1
11 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY49087426654774581266832110', '511722 924833 2244',
12 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856', '7
13 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '3784 662233 17389',
14 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951',
15 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279566 77631', '1
16 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451556 48766', '1
17 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143310514852179535', '455676 6437463635', '5969
18 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '4834'
19 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3057', 'LU931822574697545215', '3484 621767 21237', '6805', '1
20 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3064', 'PS146965545449253377627273133', '3467 732741 26810',
21 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3071', 'N08923814763512', '3464 789562 23352', '6625', '661',
22 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3078', 'IS025127145884623279548733', '4539 322 74 2377', '940
23 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3085', 'BE63114723972437', '5266 3346 1135 1687', '7241', '41
24 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3092', 'RO65LS0D1166122125447487', '3488 754223 46253', '9417
25 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3099', 'PT26105275356823705537218', '448 55418 98863 789', '5
26 • TRUNCATE INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3106', 'AT684751637751136507', '340547146305783', '0733', '30

```

Output

Action Output

| #    | Time     | Action                                                                                                   | Message           |
|------|----------|----------------------------------------------------------------------------------------------------------|-------------------|
| 4881 | 10:18:16 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9452', 'XX1860748007148... | 1 row(s) affected |
| 4882 | 10:18:16 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9453', 'XX7713298821170... | 1 row(s) affected |
| 4883 | 10:18:16 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9454', 'XX3184270063831... | 1 row(s) affected |
| 4884 | 10:18:16 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9455', 'XX5833224442116... | 1 row(s) affected |
| 4885 | 10:18:16 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9456', 'XX7412596800049... | 1 row(s) affected |

## Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

```
1 # UPDATE credit_cards
2 # SET iban = 'TR323456312213576817699999'
3 # WHERE id = 'CcU-2938';
4
5 • SELECT iban
6 FROM credit_cards
7 WHERE id = 'CcU-2938';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

| iban                       |
|----------------------------|
| CA137550525951024363769804 |

credit\_cards 4 x

Output

Action Output

| #   | Time     | Action                                                             | Message           |
|-----|----------|--------------------------------------------------------------------|-------------------|
| ✓ 1 | 12:31:07 | SELECT iban FROM credit_cards WHERE id = 'CcU-2938' LIMIT 0, 50000 | 1 row(s) returned |

Valor do iban correspondente ao id = CcU-2938 antes da modificação.

```
1 • UPDATE credit_cards
2 SET iban = 'TR323456312213576817699999'
3 WHERE id = 'CcU-2938';
4
5 • SELECT iban
6 FROM credit_cards
7 WHERE id = 'CcU-2938';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

| iban                       |
|----------------------------|
| TR323456312213576817699999 |

credit\_cards 5 x

Output

Action Output

| #   | Time     | Action                                                                            | Message                                                  |
|-----|----------|-----------------------------------------------------------------------------------|----------------------------------------------------------|
| ✓ 1 | 12:31:07 | SELECT iban FROM credit_cards WHERE id = 'CcU-2938' LIMIT 0, 50000                | 1 row(s) returned                                        |
| ✓ 2 | 12:31:42 | UPDATE credit_cards SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938' | 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 |
| ✓ 3 | 12:31:43 | SELECT iban FROM credit_cards WHERE id = 'CcU-2938' LIMIT 0, 50000                | 1 row(s) returned                                        |

Para modificar el valor del IBAN, puede utilizar la función UPDATE en la columna credit\_cards con el nuevo IBAN.

## Ejercicio 6

En la tabla "transaction" ingresa una nueva transacción con la siguiente información:

(Id: 108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit\_card\_id: CcU-9999

company\_id: b-9999

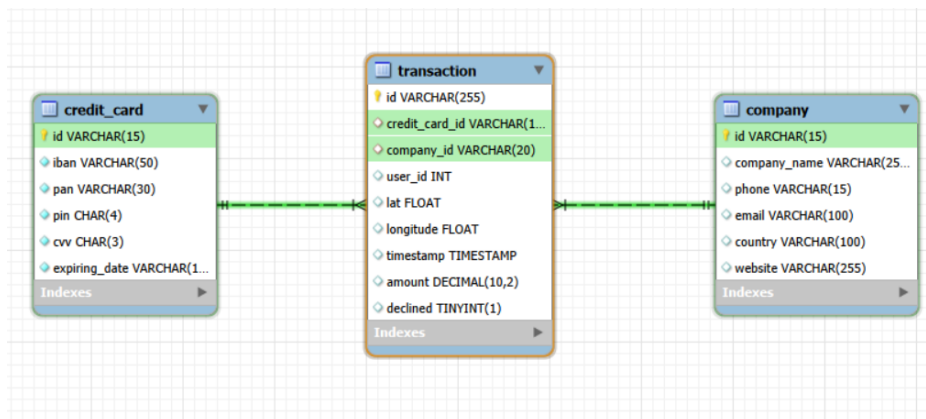
user\_id: 9999

Lat: 829.999

Longitude: -117.999

Amount: 111.11

Declined: 0)



```

1 • INSERT IGNORE INTO company (id)
2   VALUES ('b-9999');
3
4 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date)
5   VALUES (
6     'CcU-9999',
7     'ES1234567890123456789012',
8     '1234567812345678',
9     '1234',
10    '123',
11    '12/2028'
12  );
13
14 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
15   VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '111.11', '0');
16
17 • SELECT *
18   FROM transaction
19  WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';

```

| id                                   | credit_card_id | company_id | user_id | lat     | longitude | timestamp | amount | declined |
|--------------------------------------|----------------|------------|---------|---------|-----------|-----------|--------|----------|
| 108B1D1D-5B23-A76C-55EF-C568E49A99DD | CcU-9999       | b-9999     | 9999    | 829.999 | -117.999  | NULL      | 111.11 | 0        |

transaction 2 x

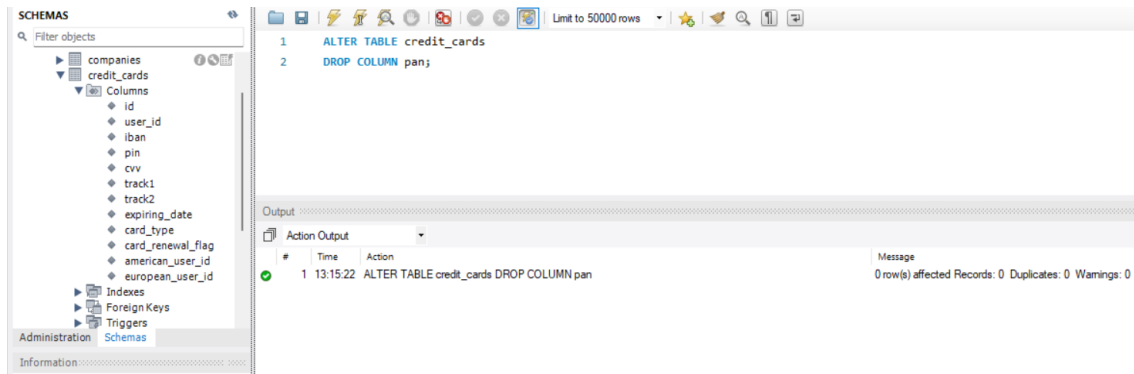
Output

| #   | Time     | Action                                                                                                     | Message           |
|-----|----------|------------------------------------------------------------------------------------------------------------|-------------------|
| ✓ 2 | 12:56:35 | INSERT IGNORE INTO company (id) VALUES ('b-9999')                                                          | 1 row(s) affected |
| ✓ 3 | 12:56:36 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ( 'CcU-9999', 'E...                | 1 row(s) affected |
| ✓ 4 | 12:56:36 | INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VAL... | 1 row(s) affected |
| ✓ 5 | 12:56:36 | SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD' LIMIT 0, 50000                 | 1 row(s) returned |

Para insertar una nueva transacción en la tabla transactions, primero es necesario verificar si ya existen datos en la tabla que coincidan con los datos que se van a insertar, utilizando la función INSERT IGNORE. De esta manera, si el ID ya existe, la consulta no funcionará; de lo contrario, se insertará el nuevo valor. Una vez hecho esto, los nuevos valores de la transacción se pueden insertar en la tabla credit\_card y, finalmente, en la tabla transactions.

## Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit\_card. Recuerda mostrar el cambio realizado.



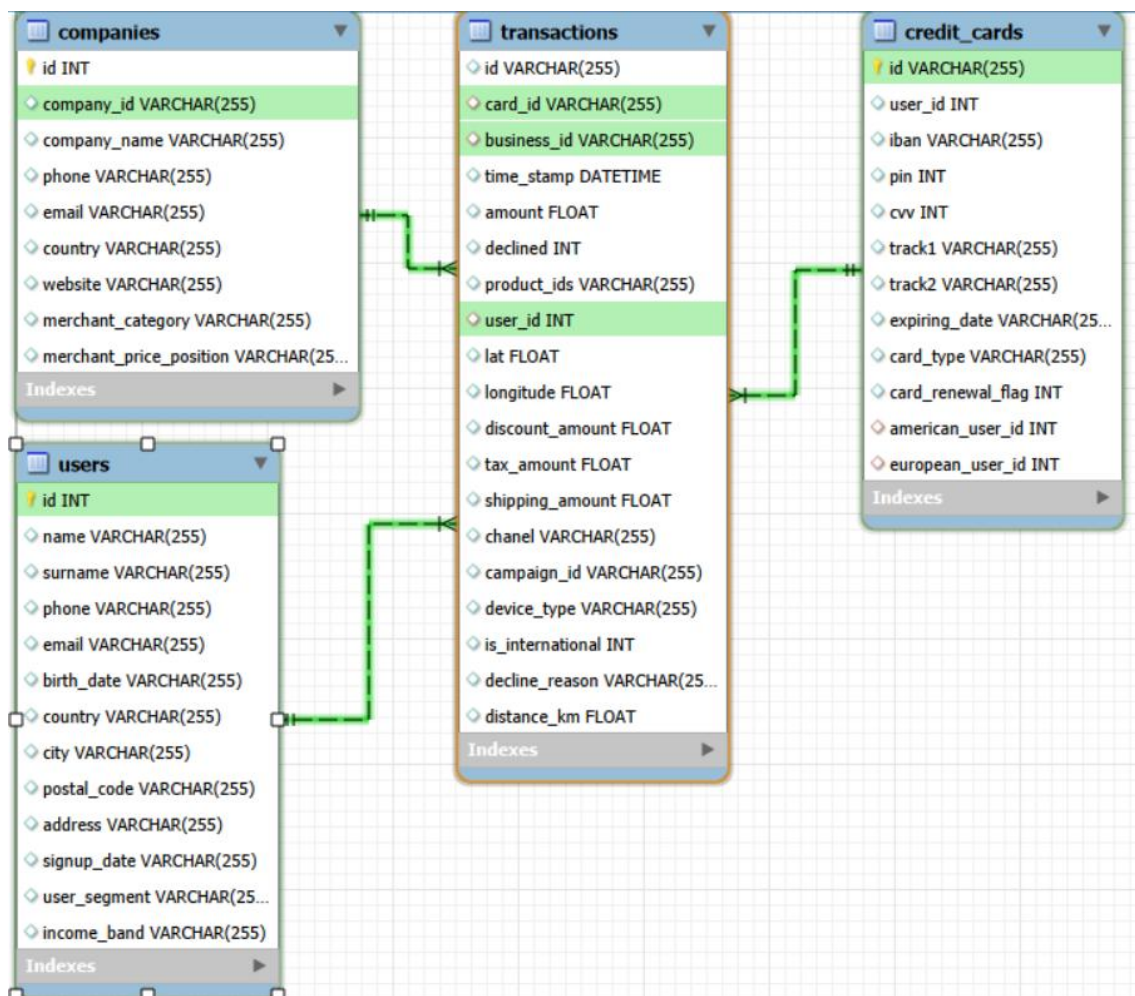
Para eliminar la columna "pan" de la tabla credit\_cards, utilice las funciones ALTER TABLE y DROP COLUMN. Esto mostrará en el menú desplegable que la columna ya no existe en la tabla.

## Ejercicio 8

Descarga los archivos CSV que **encontrarás en el apartado de recursos** :

- american\_users.csv
- european\_users.csv
- companies.csv
- credit\_cards.csv
- transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas: La tabla de products.csv la utilizaremos más adelante.



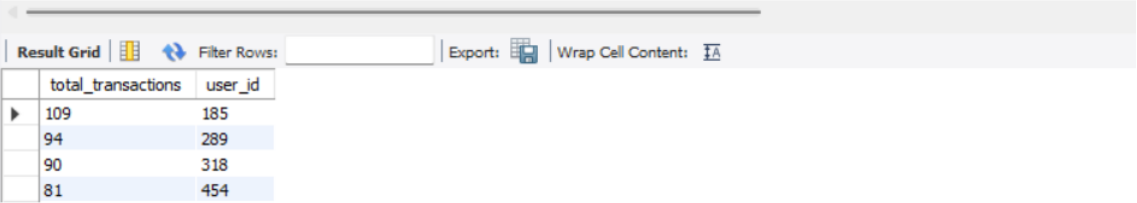
La base de datos utiliza la tabla de transacciones como tabla de hechos y las tablas de empresas, tarjetas de crédito, usuarios estadounidenses y usuarios europeos, combinadas en una única tabla de usuarios, como tablas de dimensiones. Estas tablas se relacionan mediante sus claves primarias, company\_id para empresas, id para usuarios e id para tarjetas de crédito, con las claves foráneas correspondientes presentes en la tabla de transacciones. Por lo tanto, utilizando el recurso de ingeniería inversa de MySQL, las relaciones y el esquema en estrella de la base de datos pueden verificarse visualmente.



## Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
1 • SELECT COUNT(t.id) AS total_transactions, t.user_id
2   FROM transactions t
3  WHERE t.user_id IN (SELECT U.id FROM users u) AND t.declined = 0
4   GROUP BY t.user_id
5   HAVING total_transactions > 80 AND user_id IS NOT NULL
6   ORDER BY total_transactions DESC;
```



|   | total_transactions | user_id |
|---|--------------------|---------|
| ▶ | 109                | 185     |
|   | 94                 | 289     |
|   | 90                 | 318     |
|   | 81                 | 454     |

Result 4 ×

Output

Action Output

| #   | Time     | Action                                                                                                 | Message           |
|-----|----------|--------------------------------------------------------------------------------------------------------|-------------------|
| ✓ 1 | 13:29:07 | SELECT COUNT(t.id) AS total_transactions, t.user_id FROM transactions t WHERE t.user_id IN (SELECT ... | 4 row(s) returned |

Para encontrar usuarios con más de 80 transacciones, puede utilizar una subconsulta que filtre los ID de usuario relacionando sus ID con las transacciones que han realizado y, a continuación, filtrando aquellos que tengan un total de más de 80 transacciones.



## Ejercicio 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
1 • SELECT ROUND(AVG(t.amount),2) AS promedio, cr.iban, co.company_name
2 FROM transactions AS t
3 INNER JOIN credit_cards AS cr ON t.card_id = cr.id
4 INNER JOIN companies AS co ON t.business_id = co.company_id
5 WHERE co.company_name = 'Donec Ltd'
6 GROUP BY cr.iban;
```

| promedio | iban                       | company_name |
|----------|----------------------------|--------------|
| 361.01   | CA901355773586771626108990 | Donec Ltd    |
| 142.56   | US729212642816955651428791 | Donec Ltd    |
| 246.9    | US899450803230933276819251 | Donec Ltd    |
| 154.01   | US581097432668325541142297 | Donec Ltd    |
| 250.51   | CA441226267105857473570574 | Donec Ltd    |
| 203.95   | CA332171962352073015086775 | Donec Ltd    |
| 438.16   | US985359628701198880628720 | Donec Ltd    |
| 520.37   | US788111972997693675524914 | Donec Ltd    |
| 204.63   | CA867886529258577442049898 | Donec Ltd    |

Result 1 x

Output

Action Output

| # | Time     | Action                                                                                           | Message             |
|---|----------|--------------------------------------------------------------------------------------------------|---------------------|
| 1 | 13:31:10 | SELECT ROUND(AVG(t.amount),2) AS promedio, cr.iban, co.company_name FROM transactions AS t IN... | 371 row(s) returned |

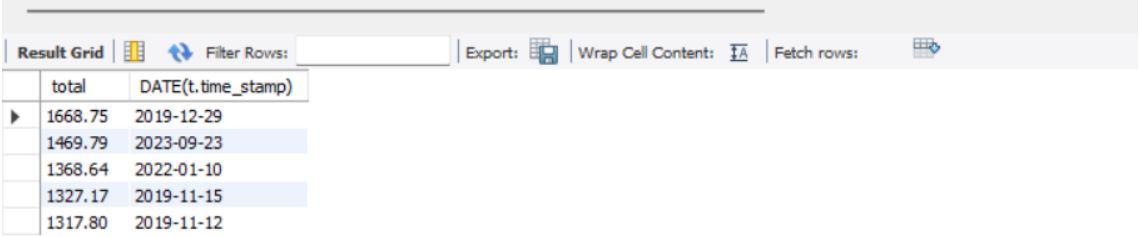
Para calcular el importe medio por IBAN de las tarjetas de crédito de la empresa Donec, se necesitan dos uniones (JOIN). La primera une la tabla credit\_cards con la tabla transactions, y la segunda une la tabla companies con la tabla transactions. Esto permite calcular el importe medio mediante la función AVG() y, a continuación, utilizar la cláusula WHERE para filtrar las empresas que se llaman Donec.

## Nivel 2

### Ejercicio 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

```
1 • SELECT CAST(SUM(t.amount) AS DECIMAL(10,2)) AS total, DATE(t.time_stamp)
2   FROM transactions AS t
3   INNER JOIN companies AS co ON co.company_id = t.business_id
4   WHERE t.declined = 0
5   GROUP BY t.time_stamp
6   ORDER BY total DESC
7   LIMIT 5;
```



The screenshot shows a database interface with a query editor and a result grid. The query is a SQL statement that calculates the total sales for each day, filtered by 'declined = 0', and orders the results by total sales in descending order, limiting the output to 5 rows. The result grid displays two columns: 'total' and 'DATE(t.time\_stamp)'. The data is as follows:

| total   | DATE(t.time_stamp) |
|---------|--------------------|
| 1668.75 | 2019-12-29         |
| 1469.79 | 2023-09-23         |
| 1368.64 | 2022-01-10         |
| 1327.17 | 2019-11-15         |
| 1317.80 | 2019-11-12         |

Below the result grid, there is an 'Output' section showing the execution of the query. It includes a table with columns '#', 'Time', 'Action', and 'Message'. The first row shows the query execution at 13:31:01, returning 5 rows.

| # | Time     | Action                                                                                               | Message           |
|---|----------|------------------------------------------------------------------------------------------------------|-------------------|
| 1 | 13:31:01 | SELECT CAST(SUM(t.amount) AS DECIMAL(10,2)) AS total, DATE(t.time_stamp) FROM transactions AS t I... | 5 row(s) returned |

Dado que los valores de la columna time\_stamp están en formato AAAA-MM-DD hh:mm:ss, para evitar errores se utiliza la función DATE() para extraer únicamente la fecha en formato AAAA-MM-DD. Así, agrupando por time\_stamp y realizando una unión de la tabla de transacciones con la de empresas, podemos obtener, mediante la función SUM(), el número total de entradas por día. De este modo, ordenando los valores totales y limitando la impresión a 5 valores en orden descendente, obtenemos los 5 días con mayor venta de entradas.

## Ejercicio 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

```
1 • SELECT co.company_name, co.phone, co.country, DATE(t.time_stamp), t.amount
2 FROM companies AS co
3 JOIN transactions AS t
4 ON t.business_id = co.company_id
5 WHERE t.amount >350 AND t.amount <400 AND
6 DATE(time_stamp) IN('2015-04-29', '2018-07-20', '2024-03-13')
7 ORDER BY amount ASC;
```

| company_name           | phone            | country       | DATE(t.time_stamp) | amount |
|------------------------|------------------|---------------|--------------------|--------|
| Tempor Diam Institute  | +89 848 141 3136 | Netherlands   | 2024-03-13         | 350.71 |
| Netus Et Malesuada Ltd | +52 818 402 1638 | Netherlands   | 2024-03-13         | 352.21 |
| Pede Cum Ltd           | +26 625 798 6126 | Norway        | 2018-07-20         | 358.64 |
| Mauris Institute       | +75 473 619 9472 | Sweden        | 2015-04-29         | 369.2  |
| Auctor Mauris Vel LLP  | +32 133 447 8239 | United States | 2024-03-13         | 383.58 |
| Frinqilla LLC          | +1 150 784 1716  | New Zealand   | 2015-04-29         | 390.33 |

Result 1 x

Output

Action Output

| # | Time     | Action                                                                                             | Message           |
|---|----------|----------------------------------------------------------------------------------------------------|-------------------|
| 1 | 10:25:48 | SELECT co.company_name, co.phone, co.country, DATE(t.time_stamp), t.amount FROM companies AS co... | 6 row(s) returned |

Para mostrar los datos de las empresas que realizaron transacciones con valores entre 350 y 400 euros el 29 de abril de 2015, el 20 de julio de 2018 y el 13 de marzo de 2024, se puede utilizar una simple operación JOIN entre las tablas de empresas y transacciones, seguida de un filtro compuesto que limita el valor de la transacción a los límites establecidos y garantiza que las fechas correspondan a los tres días especificados.

## Ejercicio 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de 400 transacciones o menos.

```
1 • SELECT co.company_name, COUNT(t.id) AS transacciones,
2 CASE
3     WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400 transacciones'
4     ELSE 'Menos de 400 transacciones'
5 END AS clasificacion_ventas
6 FROM companies AS co
7 JOIN transactions AS t ON co.company_id = t.business_id
8 WHERE t.declined = 0
9 GROUP BY co.company_id, co.company_name
10 ORDER BY transacciones;
```

| company_name                       | transacciones | clasificacion_ventas             |
|------------------------------------|---------------|----------------------------------|
| ► Lorem Eu Incorporated            | 374           | Menos de 400 transacciones       |
| ► Nec Luctus LLC                   | 391           | Menos de 400 transacciones       |
| ► Fringilla LLC                    | 392           | Menos de 400 transacciones       |
| ► Dui Quis Institute               | 397           | Menos de 400 transacciones       |
| ► Risus Associates                 | 404           | Igual o más de 400 transacciones |
| ► Amet Luctus Vulputate Foundation | 406           | Igual o más de 400 transacciones |
| ► Mattis Foundation                | 407           | Igual o más de 400 transacciones |

Result 2 x

Output

Action Output

| #   | Time     | Action                                                                                           | Message             |
|-----|----------|--------------------------------------------------------------------------------------------------|---------------------|
| ✓ 1 | 08:15:33 | SELECT co.company_name, COUNT(t.id) AS transacciones, CASE WHEN COUNT(t.id) >= 400 THEN 'Igua... | 100 row(s) returned |

Una forma de filtrar las empresas con menos o más de 400 transacciones es mediante la función CASE, donde la condición WHEN se utiliza para filtrar el recuento de ID de transacción si el valor es igual o superior a 400, y la condición ELSE para los valores restantes. De esta manera, al realizar una unión entre las tablas transactions y companies, se puede agrupar por el ID de cada empresa, obteniendo una tabla con las empresas clasificadas según su volumen de transacciones.

## Ejercicio 4

Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

Antes de la eliminación:

```
1 • SELECT * FROM world_transactions.transactions
2 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

| id                                   | card_id  | business_id | time_stamp          | amount | declined | product_ids | user_id | lat     | longitude | discount_amount |
|--------------------------------------|----------|-------------|---------------------|--------|----------|-------------|---------|---------|-----------|-----------------|
| 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD | CcS-5019 | b-2370      | 2016-12-21 20:07:00 | 172.56 | 0        | 66, 69, 87  | 438     | 41.5972 | 12.2218   | 19.92           |

transactions 3 x

Output

Action Output

| # | Time     | Action                                                                                         | Message           |
|---|----------|------------------------------------------------------------------------------------------------|-------------------|
| 1 | 11:47:01 | SELECT * FROM world_transactions.transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1C... | 1 row(s) returned |

Registro eliminado:

```
1 • DELETE FROM transactions
2 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Output

Action Output

| # | Time     | Action                                                                     | Message           |
|---|----------|----------------------------------------------------------------------------|-------------------|
| 1 | 13:29:38 | DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' | 1 row(s) affected |

Consulta del registro:

```
1 • SELECT * FROM world_transactions.transactions
2 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

| id | card_id | business_id | time_stamp | amount | declined | product_ids | user_id | lat | longitude | discount_amount | tax_amount | shipping_amount | channel |
|----|---------|-------------|------------|--------|----------|-------------|---------|-----|-----------|-----------------|------------|-----------------|---------|
|----|---------|-------------|------------|--------|----------|-------------|---------|-----|-----------|-----------------|------------|-----------------|---------|

transactions 4 x

Output

Action Output

| # | Time     | Action                                                                                         | Message           |
|---|----------|------------------------------------------------------------------------------------------------|-------------------|
| 1 | 11:49:42 | SELECT * FROM world_transactions.transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1C... | 0 row(s) returned |

## Ejercicio 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

```
1 • CREATE VIEW VistaMarketing AS
2   SELECT co.company_name, co.phone, co.country, media_de_compra
3   FROM companies AS co
4   JOIN (
5     SELECT business_id, ROUND(AVG(t.amount),2) AS media_de_compra
6     FROM transactions AS t
7     GROUP BY business_id
8   ) t ON t.business_id = co.company_id
9   ORDER BY media_de_compra DESC;
10 • SELECT * FROM VistaMarketing;
```

| company_name                      | phone            | country        | media_de_compra |
|-----------------------------------|------------------|----------------|-----------------|
| Ac Fermentum Incorporated         | +64 601 244 6320 | Germany        | 330.77          |
| Ut Semper Foundation              | +10 283 500 6449 | Sweden         | 307.37          |
| Orci Adipiscing Limited           | +60 897 390 5820 | United Kingdom | 306.27          |
| Dolor Vitae Limited               | +6 421 567 9195  | France         | 305.95          |
| Nunc In Foundation                | +13 267 741 2619 | Italy          | 305.4           |
| Maecenas Malesuada Fringilla Inc. | +51 673 540 5289 | Netherlands    | 305.15          |
| Aliquet Sem Limited               | +44 744 425 9915 | Netherlands    | 304.21          |
| Nulla Integer Vulputate Corp.     | +75 236 687 9162 | Sweden         | 303.42          |
| Non Justo Corp.                   | +67 963 294 1666 | Sweden         | 303.22          |
| Turpis Company                    | +17 624 311 7168 | Netherlands    | 302.77          |
| Aliquam Erat Volutpat LLP         | +39 372 330 7229 | Italy          | 301.97          |
| Amet Lorem LLP                    | +40 468 847 7738 | Spain          | 301.82          |
| Placerat LLP                      | +17 324 602 5033 | Netherlands    | 301.61          |

VistaMarketing 4 x

Output

Action Output

| #   | Time     | Action                                                                                          | Message             |
|-----|----------|-------------------------------------------------------------------------------------------------|---------------------|
| ✓ 1 | 11:53:27 | CREATE VIEW VistaMarketing AS SELECT co.company_name, co.phone, co.country, media_de_compra ... | 0 row(s) affected   |
| ✓ 2 | 11:53:27 | SELECT * FROM VistaMarketing LIMIT 0, 50000                                                     | 100 row(s) returned |

Una forma de proporcionar al departamento de marketing la información solicitada es mediante la creación de una vista CREATE VIEW. Para ello, se utiliza una subconsulta con las funciones AVG y ROUND para calcular el promedio de compra, agrupando los datos por el ID de cada negocio. De esta manera, al realizar una unión JOIN entre la tabla de empresas companies y esta subconsulta, se integran los datos de contacto con la media calculada, obteniendo una tabla virtual consolidada y ordenada de mayor a menor según su promedio de compras.

## Nivel 3

### Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

👉 ¿Cuántas tarjetas están activas?

```
1 • WITH UltimasTransacciones AS (  
2     SELECT card_id, declined,  
3         ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY time_stamp DESC) AS rn  
4     FROM transactions  
5 ),  
6 EstadoTarjetas AS (  
7     SELECT card_id, SUM(declined) AS total_rechazos, COUNT(*) AS total_transacciones  
8     FROM UltimasTransacciones  
9     WHERE rn <= 3  
10    GROUP BY card_id  
11 ),  
12 TablaFinal AS (  
13     SELECT card_id,  
14         CASE WHEN total_transacciones = 3 AND total_rechazos = 3 THEN 'inactivo'  
15         ELSE 'activo'  
16         END AS estado  
17     FROM EstadoTarjetas  
18 )  
19 SELECT COUNT(*) AS tarjetas_activas  
20 FROM TablaFinal  
21 WHERE estado = 'activo';
```

Result Grid

| tarjetas_activas |
|------------------|
| 5000             |

Result 13 x

Output

Action Output

| #   | Time     | Action                                                                                  | Message           |
|-----|----------|-----------------------------------------------------------------------------------------|-------------------|
| ✓ 1 | 18:24:21 | WITH UltimasTransacciones AS ( SELECT card_id, declined, ROW_NUMBER() OVER (PARTITIO... | 1 row(s) returned |

El uso de la cláusula WITH permite estructurar la consulta mediante Expresiones de Tabla Comunes, actuando como tablas temporales que dividen el código en bloques lógicos para facilitar su comprensión y análisis paso a paso. Por su parte, la cláusula PARTITION BY se emplea dentro de las funciones de ventana para segmentar los datos en grupos específicos, en este caso, por el ID de cada tarjeta. De esta manera, al combinarse con la función ROW\_NUMBER, se logra reiniciar el conteo para cada grupo, lo que permite enumerar y ordenar las transacciones de forma cronológica e independiente para cada tarjeta.

## Ejercicio 2

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product\_ids. Genera la siguiente consulta:

👉 Necesitamos conocer el número de veces que se ha vendido cada producto.

```
1 • DROP TABLE IF EXISTS product_sales;
2
3 CREATE TABLE product_sales AS
4 SELECT
5     t.id AS transaction_id,
6     jt.id_producto AS product_id
7 FROM transactions t
8 CROSS JOIN JSON_TABLE(
9     CAST(CONCAT('[', t.product_ids, ']') AS JSON),
10    '$[*]' COLUMNS(id_producto INT PATH '$')
11 ) jt
12 WHERE t.product_ids IS NOT NULL
13     AND t.product_ids != '';
14
15 • SELECT * FROM product_sales;
```

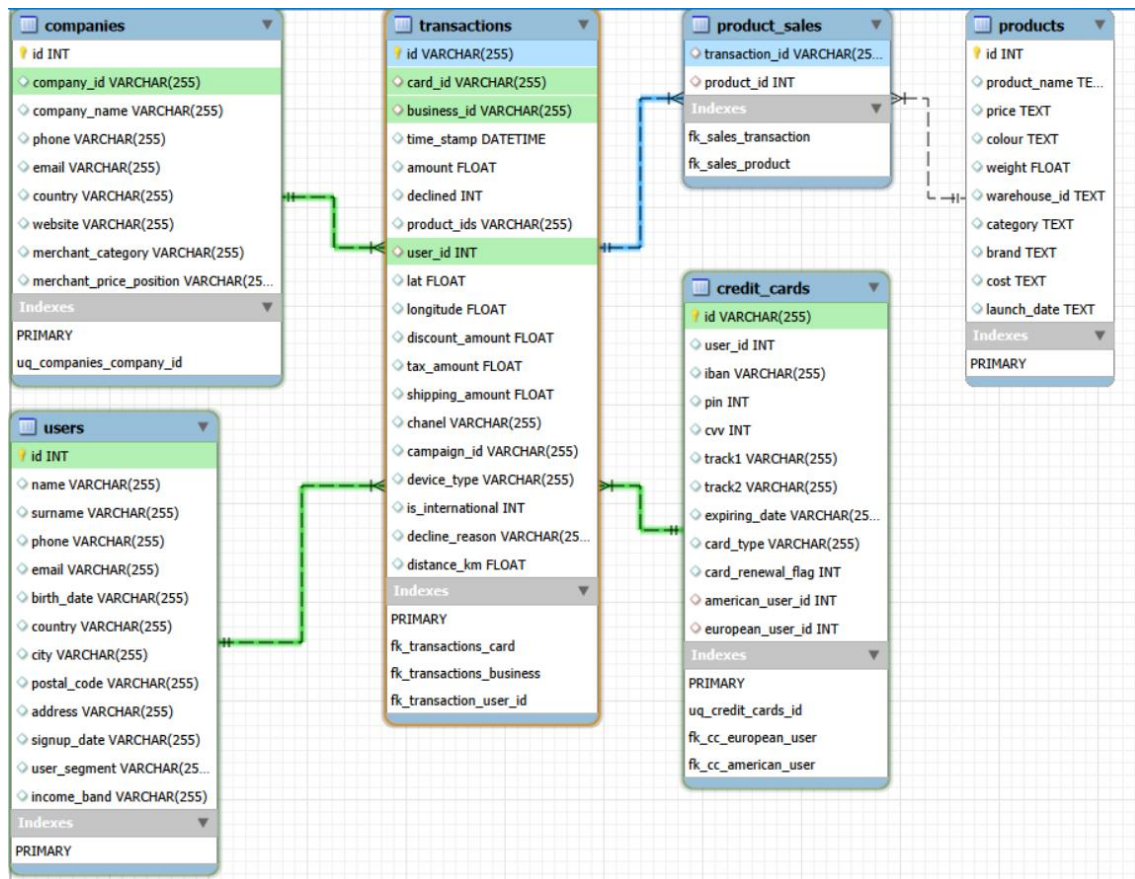
| transaction_id                       | product_id |
|--------------------------------------|------------|
| 00043A49-2949-494B-A5DD-A5BAE3B819DD | 16         |
| 00043A49-2949-494B-A5DD-A5BAE3B819DD | 26         |
| 00043A49-2949-494B-A5DD-A5BAE3B819DD | 97         |
| 00043A49-2949-494B-A5DD-A5BAE3B819DD | 87         |
| 00045D68-ED2E-4F2F-8186-CEE074D875D0 | 30         |
| 00045D68-ED2E-4F2F-8186-CEE074D875D0 | 11         |
| 00045D68-ED2E-4F2F-8186-CEE074D875D0 | 16         |
| 00045D68-ED2E-4F2F-8186-CEE074D875D0 | 81         |
| 000481C3-1C26-4FEF-83A0-4CD0EB004B8D | 72         |
| 00051AA4-9CBE-4268-B070-C38062A1B3E2 | 18         |
| 0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D | 35         |

product\_sales 2 x

Output

| # | Time     | Action                                                                                         | Message                                                          |
|---|----------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 1 | 12:57:57 | DROP TABLE IF EXISTS product_sales                                                             | 0 row(s) affected                                                |
| 2 | 12:57:57 | CREATE TABLE product_sales AS SELECT t.id AS transaction_id, jt.id_producto AS product_id F... | 253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0 |
| 3 | 12:58:00 | SELECT * FROM product_sales LIMIT 0, 50000                                                     | 50000 row(s) returned                                            |





Una forma de conectar los datos de los productos con las transacciones es mediante la creación de una tabla intermedia `product_sales` utilizando la función `JSON_TABLE` junto con `CROSS JOIN`. Esta técnica permite transformar la lista de IDs almacenada como texto, convirtiéndola temporalmente a formato JSON, para desglosarla en múltiples filas individuales. De esta manera, al extraer el ID de cada producto y asociarlo a su respectiva transacción, se consolida una estructura relacional óptima, como se refleja en la imagen, que resuelve la relación de muchos a muchos.

Para conocer el número exacto de veces que se ha vendido cada producto, se emplea la función de agregación `COUNT` sobre la nueva tabla intermedia. De esta manera, al realizar una unión `JOIN` con la tabla principal de productos y agrupar los resultados mediante `GROUP BY`, se obtiene un listado claro y detallado con el volumen de ventas de cada artículo, clasificado de mayor a menor.